

User Mode Linux Redesign

Architecture, Implementation State, and Validation Evidence

Reporting cutoff:

`umlctl-deploy @ 485eff5c96be`

May 14, 2026

Goal and State

UML can become a profile-driven kernel research substrate again.

The redesign turns one source tree into fast production, observable research, fuzzing, sandbox, library, and time-travel artifacts.

backend abstraction

static-key hot paths

profile composition

Phase J validation

Original promise

- Linux kernel as a userspace process
- rootless development and debugging
- isolated Linux instances
- no machine emulation requirement

Modern blockers

- userspace trap cost
- SMP and state ownership
- fuzzing throughput
- stale profile and tooling story

RESPONSE

backend abstraction + runtime gates + profile matrix + kvm-v2

Build from one tree

- near-native prod-fast
- sanitizer-heavy research
- KCOV/snapshot fuzz
- minimized sandbox
- direct-call library
- deterministic time-travel

Do not build

- a Linux reimplementation
- a QEMU/Firecracker replacement
- a container runtime
- a from-scratch hypervisor

Current State at May 14, 2026

Cutoff	485eff5c96be on umlctl-deploy
Functional headline	kvm-v2 cleared Phases A–H, G.1/G.2, and I; Phase J is active
Parity	cpython-parity 21/21 under UP and SMP
Substrate gate	PASS=25/FAIL=3/XFAIL=3, matching seccomp
2h soak	kvm-v2 200/200 PASS; aggregate 397/400 PASS
Benchmark	Faster than seccomp on all eight hosts and all three tiers

DONE

backend/gates/profile foundation

IN PROGRESS

Phase J validation

BLOCKED

large upstream series until validation

21/21

cpython parity
UP and SMP

200/200

kvm-v2 Phase J
two-hour soak

193–908x

micro speedup
over seccomp

7 series

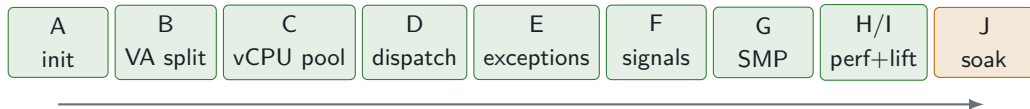
staged upstream
submission queue

Questions to Answer

- What is the revived UML value proposition?
- Why not QEMU, Firecracker, gVisor, LKL, or containers?
- What architecture lets all profiles share one tree?
- What has landed versus what remains planned?
- Does `kvm-v2` preserve SMP and architectural state?
- Does speed survive beyond microbenchmarks?
- What validation bar remains before upstream?
- What should reviewers challenge next?

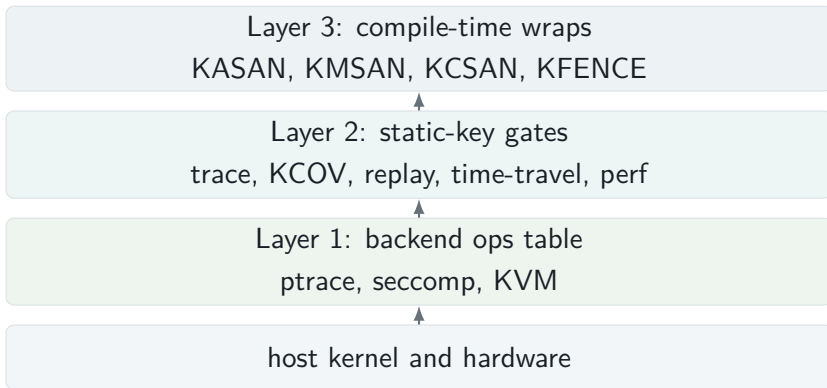
Architecture

Phase Map



- Old umbrella P0, Python C-extension import crashes, is functionally closed.
- Phase J formalizes that closure with sustained workload breadth.

Three-Layer Architecture



Layer 1: Backend Contract

Contract categories

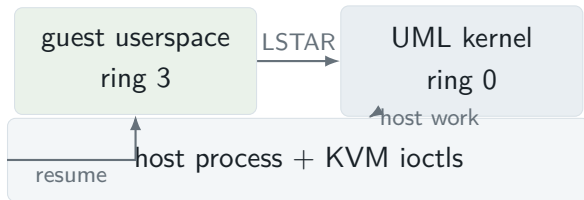
- lifecycle and trap
- memory regions
- scheduling and IPI
- time
- debug/register access

Design point

- every backend implements the table
- single-backend builds can inline calls
- dynamic builds select at boot
- capability flags replace global backend booleans

Profiles Are Product Points

Profile	Backend	Hooks	Role
prod-fast	KVM > seccomp	none	near-native production
prod-with-hooks	KVM > seccomp	compiled, off	production with emergency observability
research	seccomp	trace, kprobes	sanitizer/debug profile
fuzz	seccomp	KCOV, snapshot	syzkaller/harness fuzzing
fuzz-deep	seccomp	KCOV, replay	race and replay investigations
sandbox	seccomp-only	none compiled	minimized TCB
library	direct	n/a	LKL-style harnesses
time-travel	seccomp	deterministic time	replayable execution



This is a trap mechanism for UML, not a general-purpose VMM.

The Hard Part Was State Ownership

Audit inputs

- 149 state items
- 50+ operations
- ownership matrix
- suspect register audits
- runtime state trace

Bug classes closed

- migration while using per-CPU vCPU state
- EINTR mid-stub register recovery
- cross-task FPU/XMM leakage
- worker fd leak and shutdown hang
- AVX/XSAVE #UD drift

Validation

Validation Ladder

L0 launcher-cargo
L1 kernel-build
L2 KUnit/marshal
L3 substrate: PASS=25/FAIL=3/XFAIL=3
L4 process/syscall classes
L5 perf ratio gates
L6 real workload boot
L7 24h Phase J soak

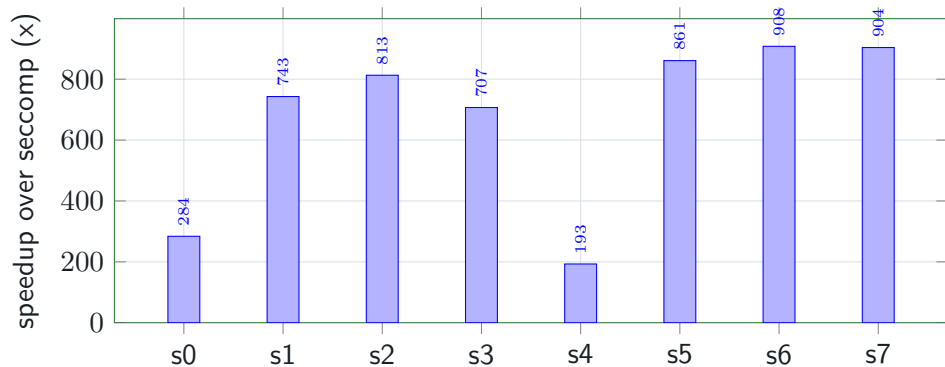
Phase J: First Sustained Run

Tuple	n	pass	fail	rate
kvm-v2 aggregate	200	200	0	100.00%
seccomp aggregate	200	197	3	98.50%
total	400	397	3	99.25%

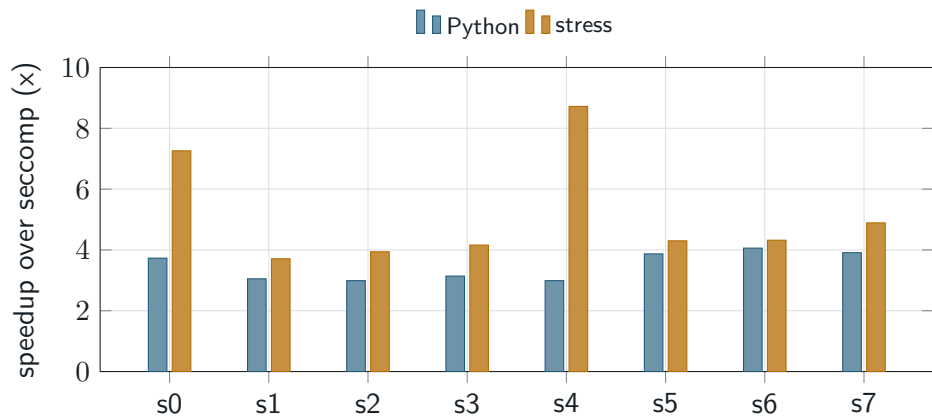
- 35 minutes wall before threshold-trip clean stop.
- Three failures were ioccheck/seccomp timeout shape, not data corruption.
- kvm-v2 passed every iteration across memcheck, ioccheck, stress-ng, and tier1-pylibs.

Performance

Micro Benchmark: The Gadget Changed the Cost Class



Python and Stress Tiers



Roadmap

What Remains

Validation

- formal 24h Phase J
- Tier 2/Tier 3 templates
- LTP curation
- wider CI matrix

Resumed work

- KVM-aware snapshot readiness
- record/replay extensions
- syzkaller vm/uml
- optional AVX-512 Phase B

Upstream Path

#	Series	Subsystem	State
1	bpf-hygiene-v1	BPF	ready
2	kmsan-arch-callback-rfc	mm/kmsan	ready
3	ftrace-notrace-generic-v1	tracing	prepared
4	backend ops abstraction RFC	arch/um	to write
5	static-key hot paths	arch/um	to write
6	per-profile C-series	arch/um + subsystems	blocked
7	KVM backend	arch/um + KVM	blocked by Phase J

The redesign is now an implementation with a validation frontier.

- The architectural shape is coherent: backends, gates, profiles.
- `kvm-v2` has crossed the hardest feasibility line: real workloads, SMP, and speed.
- The next credible milestone is not another design memo; it is Phase J completion and an upstreamable series plan.

Backup: Detailed Speedups

Host	CPU	micro	Python	stress
s0	i9-12900K	284x	3.73x	7.26x
s1	Xeon E3-1225 v6	743x	3.05x	3.71x
s2	Xeon E3-1225 v5	813x	2.99x	3.94x
s3	Xeon W-2123	707x	3.14x	4.16x
s4	i5-12600K	193x	2.99x	8.72x
s5	Ryzen 7 7840HS	861x	3.87x	4.30x
s6	Ryzen 7 7840HS	908x	4.06x	4.32x
s7	Ryzen 7 7840HS	904x	3.91x	4.89x

Backup: Source Map

- `00-vision.md`: goals, non-goals, stakeholders.
- `STATUS.md`: live state, phase ledger, blockers.
- `01-architecture/`: three layers, invariants, data flow.
- `02-workstreams/D-kvm-backend/`: KVM design, state audit, benchmarks, Phase J.
- `03-profiles/`: profile matrix and cost model.
- `tools/testing/selftests/um/`: gates, benchmarks, soak, parity.
- `tools/uml/uml-launcher/`: launcher and `umlctl`.